

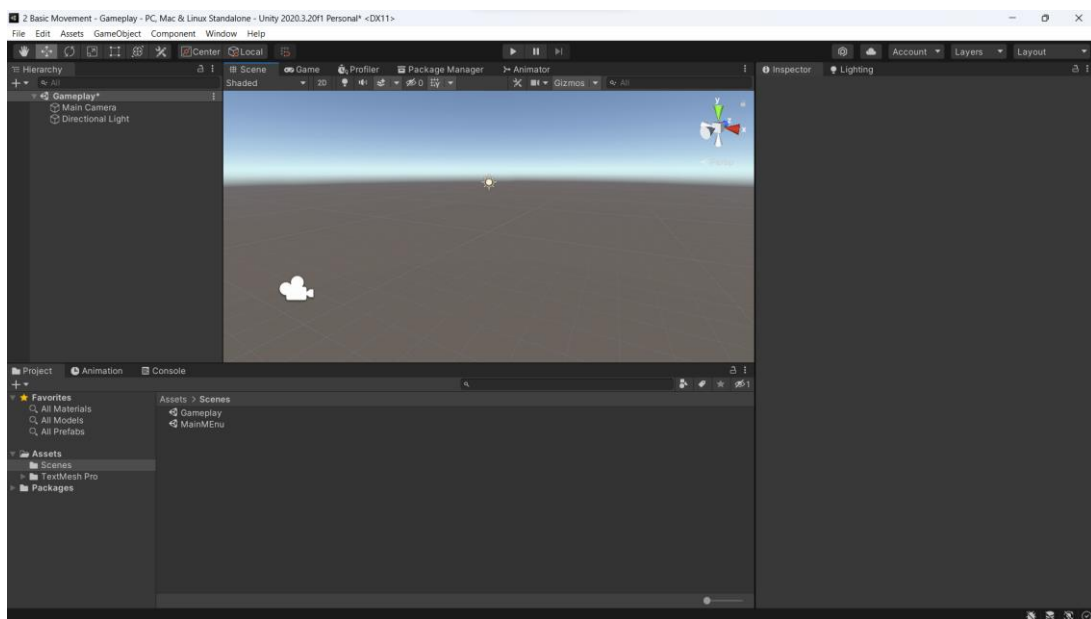
MODUL 2

PRAKTIKUM PEMROGRAMAN MULTIMEDIA & GAME

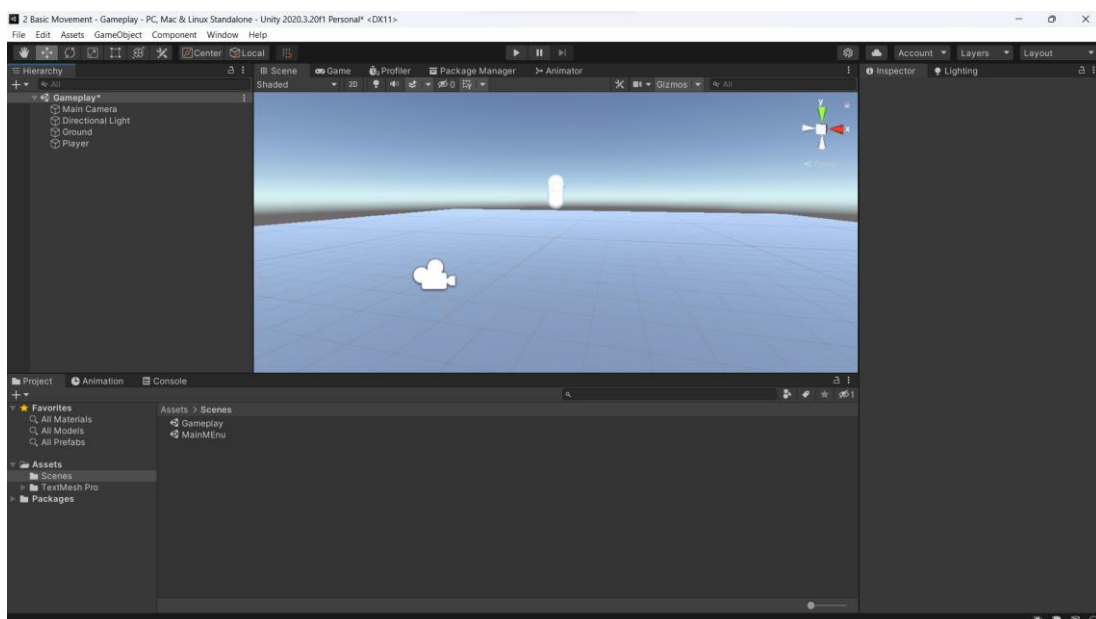
Basic Movement

Pada Modul ini, Mahasiswa akan mempelajari tentang development basic movement pada karakter game. Terdapat banyak sekali metode untuk membuat movement karakter, tetapi pada modul ini, mahasiswa akan mempelajari pemrograman movement menggunakan aturan fisika computer. Berikut adalah Langkah Langkah pembuatanya.

1. Buka Project Unity Modul 1 anda.



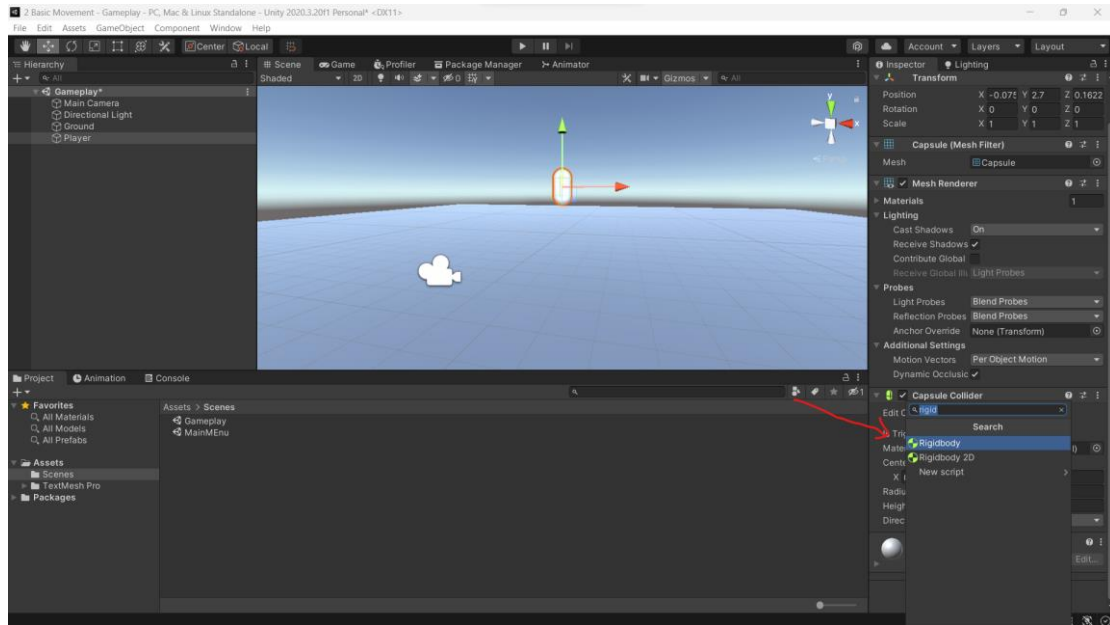
2. Buat 2 Game Object baru, Plane dan Capsule.



Buat 2 game object baru.

- Plane : sebagai lantai/ground, buat skala plane menjadi seluas Digambar.
- Capsule : yang akan berfungsi sebagai player.

3. Tambahkan Component Rigidbody pada Player.

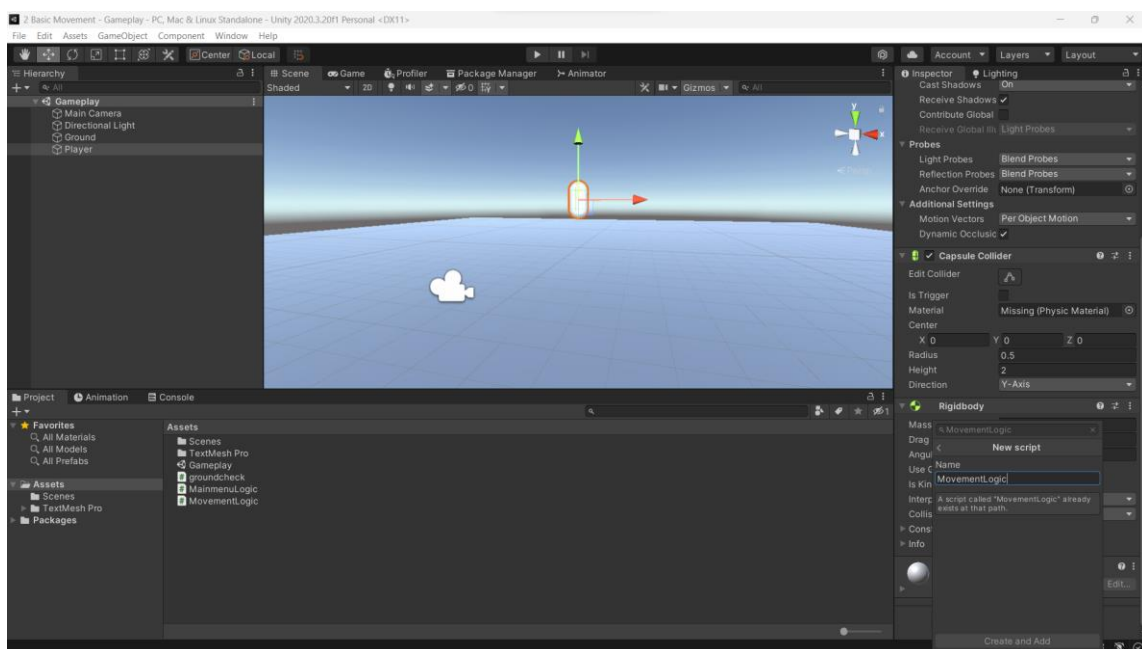


Seleksi Player – Inspector – add Component – Rigidbody.

Setelah itu masuk ke play mode, dan amati perbedaan player Ketika sebelum ditambahkan component rigidbody, dan sesudah ditambahkan component Rigidbody.

Rigidbody adalah Component yang akan menambahkan sifat fisika kepada GameObject.

4. Tambahkan Component new Script bernama MovementLogic



5. Memprogram Movement Karakter.

Tambahkan beberapa Variable Global sebagai berikut.

```
private Rigidbody rb;
public float walkspeed = 0.5f;
private Transform PlayerOrientation;
float horizontalInput;
float verticalInput;
Vector3 moveDirection;
```

Lakukan inialisasi Variable Rigidbody di method start.

```
Unity Message | 0 references
void Start()
{
    rb = this.GetComponent<Rigidbody>();
    PlayerOrientation = this.GetComponent<Transform>();
}
```

Tuliskan Method Movement untuk logika Movement karakter.

```
Unity Message | 0 references
void Update()
{
    Movement();
}

1 reference
private void Movement()
{
    horizontalInput = Input.GetAxisRaw("Horizontal");
    verticalInput = Input.GetAxisRaw("Vertical");

    moveDirection = PlayerOrientation.forward * verticalInput + PlayerOrientation.right * horizontalInput;
    rb.AddForce(moveDirection.normalized * walkspeed * 10f, ForceMode.Force);
}
```

Setelah penulisan Selesai, Save dan Kembali ke unity. Pastikan tidak ada error, jika masih terdapat error, lakukan pengecekan ulang terhadap script.

6. Lakukan Percobaan Movement karakter.

Masuk ke Play mode, dan tekan tombol WASD atau ↑ ↓ → ← untuk menggerakkan karakter. Jika karakter bisa bergerak, Script yang anda tulis sudah benar. Jika belum, lakukan pengecekan ulang!

7. Pembuatan Logika Running.

Update Beberapa Script yang sudah ditulis. Pertama adalah penambahan Variable Global.

```
private Rigidbody rb;
public float walkspeed = 0.5f, runspeed = 1f;
private Transform PlayerOrientation;
float horizontalInput;
float verticalInput;
Vector3 moveDirection;
```

Update Method Movement agar memiliki kondisi Ketika player menekan tombol **Leftshift** maka karakter akan berlari.

```
1 reference
private void Movement()
{
    horizontalInput = Input.GetAxisRaw("Horizontal");
    verticalInput = Input.GetAxisRaw("Vertical");
    moveDirection = PlayerOrientation.forward * verticalInput + PlayerOrientation.right * horizontalInput;

    if (Input.GetKey(KeyCode.LeftShift))
    {
        rb.AddForce(moveDirection.normalized * runspeed * 10f, ForceMode.Force);
    }
    else
    {
        rb.AddForce(moveDirection.normalized * walkspeed * 10f, ForceMode.Force);
    }
}
```

8. Lakukan Percobaan Movement karakter.

Masuk ke Play mode, dan tekan tombol WASD atau ↑ ↓ → ← untuk menggerakkan karakter. Dan lakukan percobaan movement dengan kombinasi tombol **LeftShift**, Jika karakter bisa berjalan dan berlari, Script yang anda tulis sudah benar. Jika belum, lakukan pengecekan ulang!

9. Selesai untuk Materi Praktikum

Materi Praktikum selesai dikerjakan jika karakter sudah dapat berjalan dan berlari.

Tugas Modul 2

Jump, Ground Detector, Fall Speed

Tugas Modul 2 adalah membuat pengembangan movement karakter. Beberapa pengembangan tersebut adalah jump, Ground Detector, Fall Speed, dan Air Manuver.

1. Penulisan Script Jump.

Dimulai dengan penambahan variable global.

```
private Rigidbody rb;
public float walkspeed = 0.5f, runspeed = 1f, jumppower = 10f;
private Transform PlayerOrientation;
float horizontalInput;
float verticalInput;
Vector3 moveDirection;
```

Dilanjutkan dengan penambahan method jump untuk penulisan logika Melompat.

```
1 reference
private void Jump()
{
    if (Input.GetKeyDown(KeyCode.Space))
    {
        Rb.AddForce(transform.up * Jumppower, ForceMode.Impulse);
    }
}
```

Setelah penulisan Script Jump, lakukan pemanggilan method Jump di method Update. Lalu save script kalian!

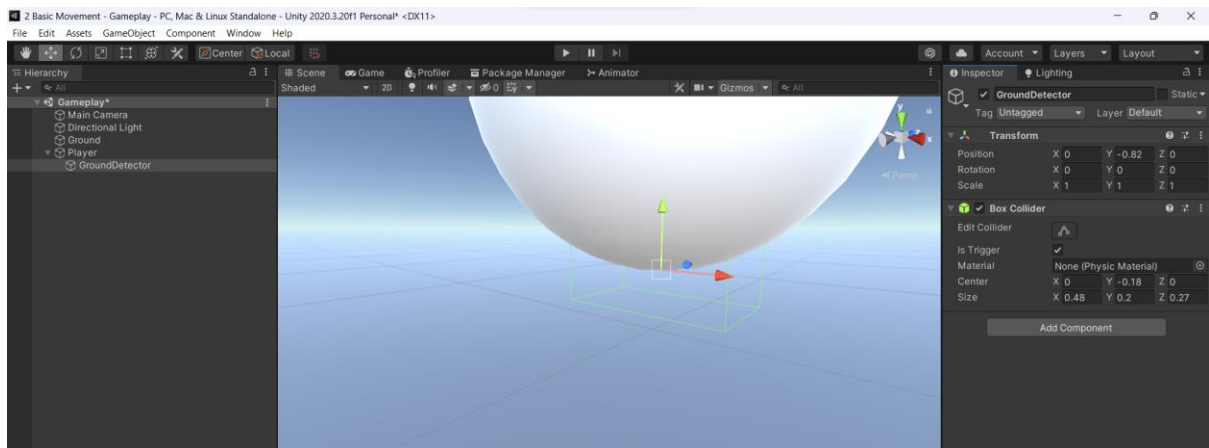
2. Lakukan percobaan Melompat.

Kembali ke unity, masuk kedalam Play mode, dan tekan tombol **Space** jika karakter bisa melompat, maka Script sudah benar, dan jika belum bisa melompat, lakukan pengecekan ulang!

3. Pembuatan Ground Detector

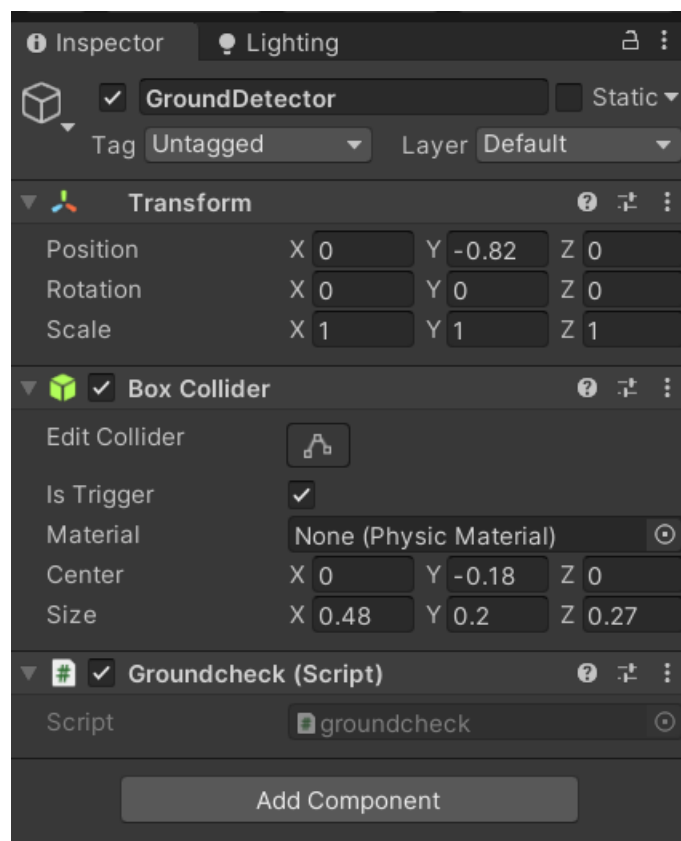
Selanjutnya adalah pembuatan Ground Detector yang berfungsi sebagai detector apakah karakter sedang menginjak Ground atau tidak. Ini juga akan berfungsi agar karakter tidak dapat melompat di udara, oleh karena itu karakter hanya akan dapat melompat Ketika sedang menginjak Ground. Pembuatan Ground

detector dapat dilakukan dengan Create empty gameobject yang diberikan component box collider seperti gambar dibawah.



Box collider itu akan berfungsi sebagai detector apakah karakter sedang menginjak Ground atau tidak.

4. Penulisan Script Ground detector.



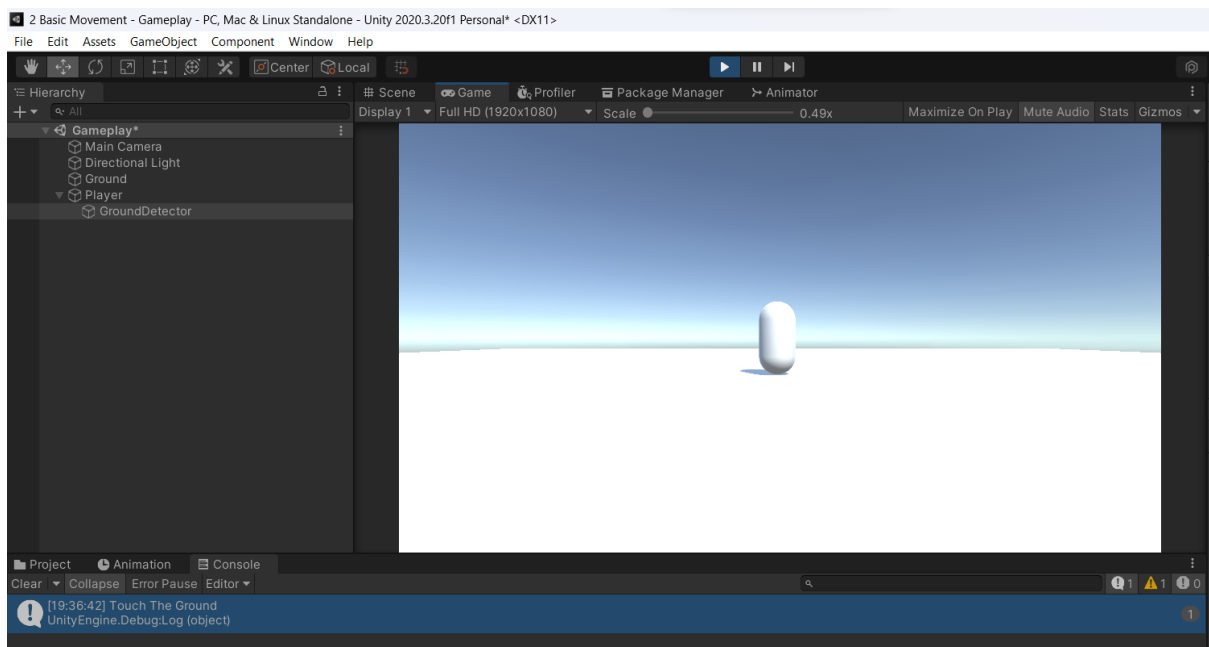
Tambahkan Component Script Groundcheck pada GameObject GroundDetector. Dan tulis script seperti di bawah ini.

```

Unity Script (1 asset reference) | 0 references
public class groundcheck : MonoBehaviour
{
    Unity Message | 0 references
    private void OnTriggerEnter(Collider other)
    {
        Debug.Log("Touch The Ground");
    }
}

```

Jika sudah, save script, dan lakukan pengujian dengan running program. jika Ketika karakter menginjak ground terdapat output console seperti gambar dibawah, maka konfigurasi sudah benar, jika tidak ada, lakukan pengecekan ulang!



5. Penyesuaian pada Script MovementLogic.cs

Selanjutnya adalah penyesuaian script MovementLogic untuk menampung input GroundCheck, dan juga mengimplementasikan pada method Jump. Pertama adalah penambahan variable global.

```

private Rigidbody rb;
public float walkspeed = 0.5f, runspeed = 1f, jumppower = 10f;
private Transform PlayerOrientation;
float horizontalInput;
float verticalInput;
Vector3 moveDirection;
bool grounded = true;

```

Setelah itu penyesuaian method jump

```

1 reference
private void Jump()
{
    if (Input.GetKeyDown(KeyCode.Space) && grounded)
    {
        rb.AddForce(transform.up * jumppower, ForceMode.Impulse);
        grounded = false;
    }
}

```

Dan penambahan method baru untuk merespon Ground Detector

```

1 reference
public void groundedchanger()
{
    grounded = true;
}

```

Terahir adalah penyesuaian script groundcheck.cs

```

Unity Script (1 asset reference) | 0 references
public class groundcheck : MonoBehaviour
{
    MovementLogic logicmovement;

    Unity Message | 0 references
    private void Start()
    {
        logicmovement = this.GetComponentInParent<MovementLogic>();
    }

    Unity Message | 0 references
    private void OnTriggerEnter(Collider other)
    {
        logicmovement.groundedchanger();
    }
}

```

Jika sudah, save program dan pastikan tidak ada error.

6. Lakukan percobaan melompat di PlayMode.

Kembali ke unity, masuk kedalam Play mode, dan tekan tombol **Space** berulang kali, jika karakter hanya dapat melompat 1x dan tidak dapat melompat Ketika ada di udara, maka Script sudah benar, dan jika karakter masih bisa melompat beberapa kali, lakukan pengecekan ulang!

7. Penulisan logika Fall Speed.

Jika diperhatikan Ketika karakter melompat, kecepatan melompat sangat cepat, namun pada saat jatuh kecepatannya sangat lambat. Selanjutnya akan dilakukan penulisan logika untuk menambah Fallspeed atau kecepatan jatuh karakter. Langkah pertama adalah penambahan variable global.

```

private Rigidbody rb;
public float walkspeed = 0.5f, runspeed = 1f, jumppower = 10f, fallspeed = 5f;
private Transform PlayerOrientation;
float horizontalInput;
float verticalInput;
Vector3 moveDirection;
bool grounded = true;

```

```

1 reference
private void Jump()
{
    if (Input.GetKeyDown(KeyCode.Space) && grounded)
    {
        rb.AddForce(transform.up * jumppower, ForceMode.Impulse);
        grounded = false;
    }
    else if (!grounded)
    {
        rb.AddForce(Vector3.down * fallspeed * rb.mass, ForceMode.Force);
    }
}

```

Jika sudah, save script, dan lakukan pengujian di play mode. Amati apakah fall speed bertambah? value dapat dimodifikasi untuk menemukan kecepatan paling sesuai.

8. Penyesuaian Jump Logic

Logika Ketika karakter sedang melompat kedepan, umumnya tidak dapat merubah arah lompatan menjadi kebelakang. Oleh karena itu akan dilakukan penyesuaian script movement agar Ketika karakter tidak menyentuh ground, input movement tidak dapat dibaca. Lakukan penyesuaian tersebut pada script movement!

9. Selesai untuk Tugas Modul 2

Jika karakter sudah dapat melompat, tidak dapat melompat di udara, menyesuaikan fall speed, dan tidak dapat berjalan Ketika di udara. Maka tugas sudah selesai.